

Algorithms and Problem Solving

Part 2:

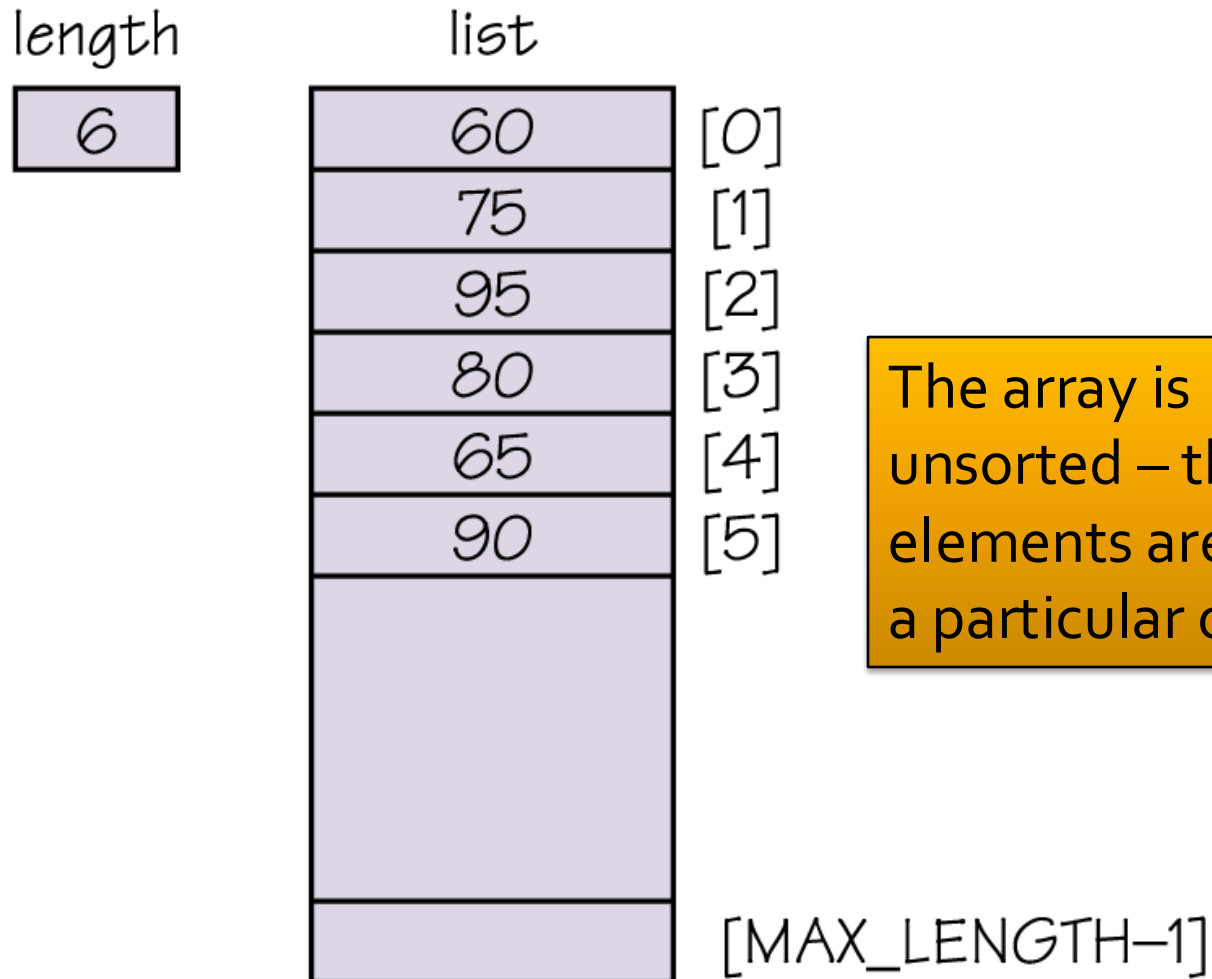
Searching and Sorting Algorithms

**CSCI 1101 Introduction to
Computer Science**

Arrays

- An **array** is a collection of related data of the same type (numbers, names, *etc.*).
- We identify each **element (or member)** of the array by its position in the array – its **index**.
- An array is really just a list.
- **Remember** – the computer can't see the whole array at once like we can, it can see only one item at a time!

An Unsorted Array



The array is
unsorted – the
elements are not in
a particular order.

Sequential Search of an Unsorted Array

- A **sequential search** examines each item in turn and compares it to the one we are searching for.
- If it matches, we have found the item. If not, we look at the next item in the array.
- We stop either when we have **found the item** or when we have looked **at all the items and not found** a match.

Sequential Search Algorithm

```
Set position to 0
Set found to FALSE
WHILE ((position < length) AND (NOT found))
    IF (list[position] equals searchitem)
        Set Found to TRUE
    ELSE
        Set position to position + 1
```

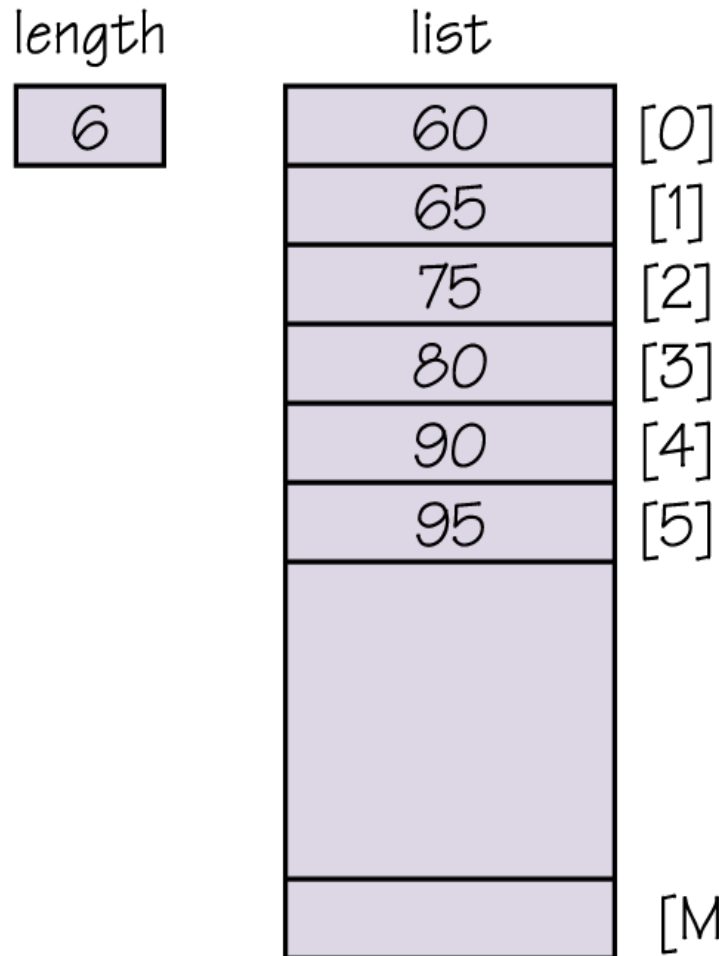
Can we improve this algorithm?

Sequential Search in a Sorted Array

- **Sorting** rearranges the elements into either ascending or descending order within the array.
- A **sorted** array is one in which the elements are in order.
- If items in an array are sorted, we can **stop looking** when we pass the place where the item would be if it were present in the array.

Is this better?

A Sorted Array



A sorted array of integers

[`MAX_LENGTH-1`]

Sequential Search of a Sorted Array

Set position to 0

Set found to FALSE

WHILE (position < length AND NOT found)

 IF (**list**[position] equals searchItem)

 Set found to TRUE

 ELSE IF (**list**[position] > searchItem)

 Set position to length

ELSE

 Set position to position + 1

Remember Divide and Conquer?

- But isn't there a better way to find stuff?
- Let's see...

“Santa's Dirty Socks” Problem:

<http://www.youtube.com/watch?v=wVPCT1VjySA>

The video introduces the class of algorithms known as divide-and-conquer.

Binary Search

Binary search (list must be sorted):

Search begins **at the middle** and **finds the item or eliminates half of the unexamined items**; process is repeated on the half where the item might be.

Ummm... What?

Binary Search

Binary search demo

<http://scratch.mit.edu/projects/2355113/>

Binary Search

```
Set first to 0
Set last to length-1
Set found to FALSE
WHILE (first <= last AND NOT found)
    Set middle to (first + last)/ 2
    IF (item equals list[middle]))
        Set found to TRUE
    ELSE
        IF (item < list [middle])
            Set last to middle – 1
        ELSE
            Set first to middle + 1
RETURN found
```

Binary Search

[0]	ant
[1]	cat
[2]	chicken
[3]	cow
[4]	deer
[5]	dog
[6]	fish
[7]	goat
[8]	horse
[9]	lion
[10]	snake

Searching for cat

First	Last	Middle	Comparison
0	10	5	cat < dog
0	4	2	cat < chicken
0	1	0	cat < ant
1	1	1	cat = cat Return: true

Searching for fish

First	Last	Middle	Comparison
0	10	5	fish > dog
6	10	8	fish < horse
6	7	6	fish = fish Return: true

Searching for zebra

First	Last	Middle	Comparison
0	10	5	zebra > dog
6	10	8	zebra > horse
9	10	9	zebra > rat
10	10	10	zebra > snake
11	10		first > last Return: false

Trace of the binary search

Binary Search Efficiency

Is a binary search
always better?

Length	Sequential Search	Binary Search
10	5.5	2.9
100	50.5	5.8
1000	500.5	9.0
10000	5000.5	12.0

Average Number of Comparisons

Sorting

- **Sorting:** Arranging items in a collection so that there is an ordering on one (or more) of the fields in the items.
- **Sort Key:** The field (or fields) on which the ordering is based.
- **Sorting algorithms:** Algorithms that order the items in the collection based on the sort key.

Why is sorting important?

Selection Example

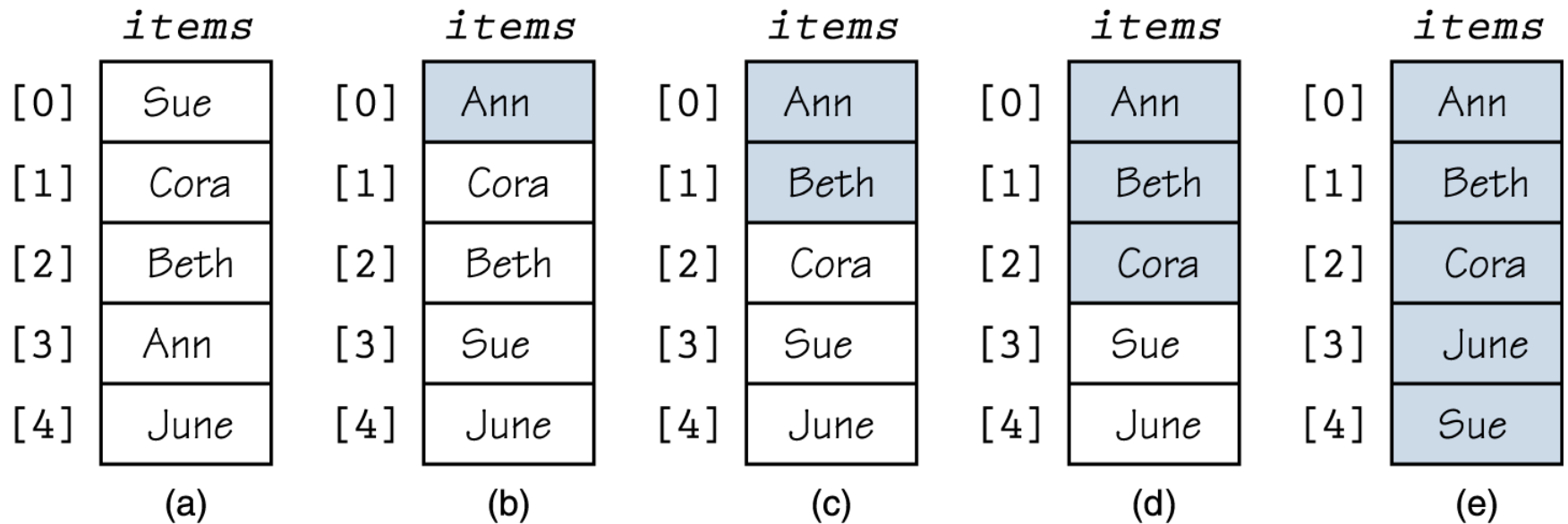
Example: Given a list of names, put them in alphabetical order.

- Find the name that comes first in the alphabet, and write it on a second sheet of paper.
- Cross out the name off the original list.
- Continue this cycle until all the names on the original list have been crossed out and written onto the second list, at which point the second list contains the same items but in sorted order.

Selection Sort

- A small modification to this manual method eliminates the need for duplicating space.
- This algorithm finds the location of the smallest element in the unsorted portion of the list, and moves it to the top of the unsorted portion of the list.
 - As you cross a name off the original list, a free space opens up.
 - Instead of writing the value found on a second list, exchange it with the value currently in the position where the crossed-off item should go.

Selection Sort



Example of selection sort (sorted elements are shaded)

Bubble Sort

- Bubble Sort is a simple comparison-based algorithm that works by repeatedly stepping through a list, comparing adjacent elements, and swapping them if they are in the wrong order. This process repeats until the list is sorted
- uses the same strategy:
 - Find the next item.
 - Put it into its proper place.
- ... But uses a different scheme for finding the next item.
- Starting with the **last** list element, compare successive pairs of elements, swapping whenever the bottom element of the pair is smaller than the one above it.

Bubble Sort

names		names		names		names		names	
[0]	Phil	[0]	Phil	[0]	Phil	[0]	Phil	[0]	Al
[1]	Al	[1]	Al	[1]	Al	[1]	Al	[1]	Phil
[2]	John	[2]	John	[2]	Bob	[2]	Bob	[2]	Bob
[3]	Jim	[3]	Bob	[3]	John	[3]	John	[3]	John
[4]	Bob	[4]	Jim	[4]	Jim	[4]	Jim	[4]	Jim

a. First iteration (Sorted elements are shaded.)

names					names					names					names				
[0]	AI	[0]	AI	[0]	AI	[0]	AI	[0]	AI	[0]	AI	[0]	AI	[0]	AI	[0]	AI	[0]	AI
[1]	Phil	[1]	Bob	[1]	Bob	[1]	Bob	[1]	Bob	[1]	Bob	[1]	Bob	[1]	Bob	[1]	Bob	[1]	Bob
[2]	Bob	[2]	Phil	[2]	Jim	[2]	Jim	[2]	Jim	[2]	Jim	[2]	Jim	[2]	Jim	[2]	Jim	[2]	Jim
[3]	John	[3]	Jim	[3]	Phil	[3]	Phil	[3]	Phil	[3]	Phil	[3]	John	[3]	John	[3]	John	[3]	John
[4]	Jim	[4]	John	[4]	John	[4]	John	[4]	John	[4]	John	[4]	Phil	[4]	Phil	[4]	Phil	[4]	Phil

b. Remaining iterations (Sorted elements are shaded.)

Sorting Summary

A highly technical demonstration of how some sorting algorithms work:

http://www.youtube.com/watch?v=INHF_5RlxTE

Time Complexity

- Computer science in its essence is applied mathematics.
- Understanding the basics of some math functions will help to get a better picture of computer science.
- Time complexity:
 - A standard way of analyzing and comparing different algorithms.
 - It allows us to find ways to improve algorithms

Logarithmic Functions

- Log (short for logarithmic) functions are commonly referred to in computer science.
- They are the **inverse of exponential functions**.

$$\log_b(M) = N \quad ==> \quad M = b^N$$

$$\log_2(8) = 3 \quad ==> \quad 2 * 2 * 2 = 2^3$$

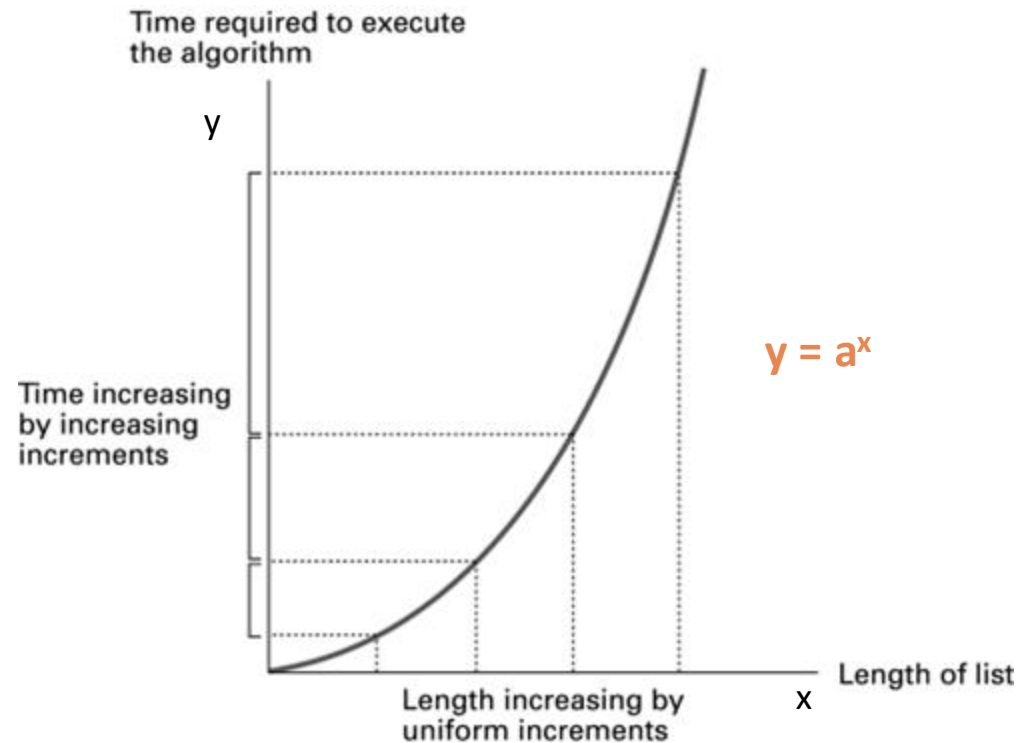
$$\log_2(16) = 4 \quad ==> \quad 2 * 2 * 2 * 2 = 2^4$$

$$\log_{10}(100) = 2 \quad ==> \quad 10 * 10 = 10^2$$

$$\log_{10}(1000) = 3 \quad ==> \quad 10 * 10 * 10 = 10^3$$

Exponential Function

They describe **rapid growth** or **decay**. As the value of x increases, the exponential function causes y to grow **more** and **more** over time

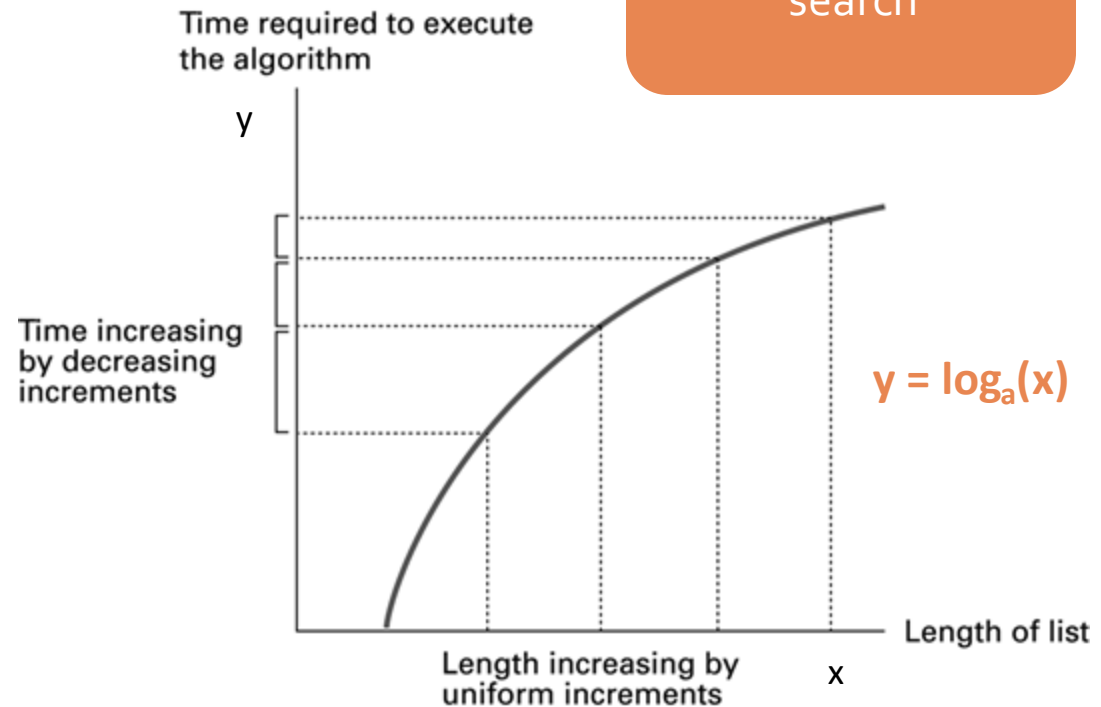


Example: Bubble Sort

Logarithmic (log) Function

They describe **slow growth**. Where an exponential function grows more and more over time, a log function grows **less and less** over time.

Example: binary search



Example

- You have 64 friends on Facebook and you want to run an algorithm that goes through each of your friends. How long is it going to take?

$$\log_b(M) = N \quad \rightarrow \quad M = b^N$$

$$\log_2(64) = 6 \quad \rightarrow \quad 2 * 2 * 2 * 2 * 2 * 2 = 64$$

Example

- What if you double the number of friends (amount of data)?

$$\log_2(128) = 7 \text{ sec} \rightarrow 2 * 2 * 2 * 2 * 2 * 2 * 2 = 128$$

- Let' double the number of friends again!

$$\log_2(264) = 8 \text{ sec} \rightarrow 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 = 264$$

Example

- What about all the people on Earth!

$$\log_2(8,589,934,594) = 33 \text{ secs}$$

- Relevance: Logarithmic functions **do not grow too much** over time. We can add as much data as we want or need, the time the algorithm will run is basically the same.

Algorithm Scalability

- We want our programs to **succeed**, so we want to make our solutions **scalable**.
- When we design an algorithm, we want to know if it will be efficient when we scale the input size.
 - Google's programs operating efficiently with 1,000 users
 - Google's programs operating efficiently with 10,000 users
 - Google's programs operating efficiently with 1,000,000 users

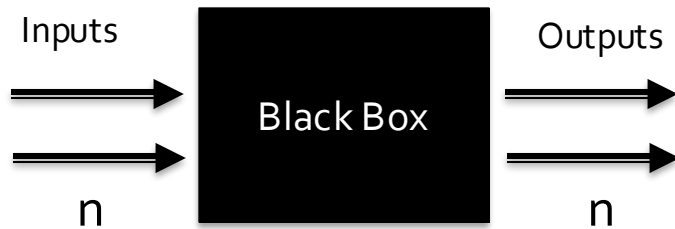
n-Notation

- n is a function of n
- That is, the output is a function of the input



n-Notation

Algorithm A



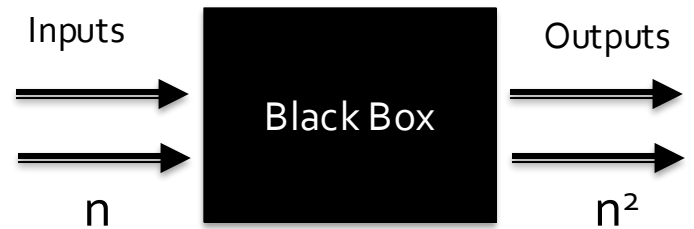
100

100

1000

1000

Algorithm B



100

$100 \times 100 = 10,000$

1000

$1000 \times 1000 = 1,000,000$

Algorithm Efficiency

- Measured as number of instructions executed.
- Big-O notation: Used to represent efficiency classes*.
 - What is the WORST the algorithm can do?
 - Example: Selection sort is in $O(n^2)$.
- Best, worst, and average case analysis.
 - Tells us how we can expect an algorithm to perform (time efficiency, space efficiency).

*Strictly speaking, it's really Big Theta, but Big-O is good enough for now (and easier to understand).